■ ANIME PERSONALITY

AI CHATBOT

BUILD Documentary

Samsung Galaxy Book5 Pro 360 · 32 GB RAM · Intel Arc iGPU → GPU Mode
Ollama + Llama 3:8B → Stheno v3.2 Q8_0 · 10-Day Sprint · 180 Prototypes

"After 179 failed prototypes, v180 is finally stable."

BUILDER	You — solo dev, self-taught, vibe-coded with Claude
DURATION	10 days (4 holidays + 6 college days)
TECH STACK	Python · Ollama · Llama 3:8B · Stheno v3.2 · Tkinter / GUI
FINAL BOTS	Rem · Miku · Yuhee · Kirito · Lelouch · Trio Chat
OUTCOME	Stable EXE app pinned to taskbar — working perfectly

CHAPTER 01 — HARDWARE REALITY CHECK

What You Had, What You Discovered

Before a single line of code was written, the machine had to be understood. The Samsung Galaxy Book5 Pro 360 looked capable on paper — but running a local LLM is a different beast entirely from ordinary laptop tasks.

LAPTOP	**Samsung Galaxy Book5 Pro 360**
RAM	**32 GB (18 GB available to Ollama — corrected from initial 18 GB misread)**
STORAGE	**1 TB NVMe**
GPU	**Intel Arc integrated GPU (iGPU)**

Key Hardware Lessons Learned

The first and most impactful technical realisation of the entire project was about where computation actually happens:

→ LLMs run best on a dedicated GPU — not on CPU.

→ Ollama skips integrated GPUs (iGPUs) by default, so at first it ran everything on the CPU — even though the laptop's Intel Arc iGPU was there all along.

→ After a key insight (and a correction on the RAM figure — ChatGPT had reportedly misled you about the 18 GB figure), GPU mode was unlocked.

→ Switching to GPU mode gave a 2×–5× speed improvement in token generation. This was a turning point for the entire project.

The lesson: always verify hardware specs independently. One wrong number (RAM available vs total) can block you from unlocking real performance.

CHAPTER 02 — THE FIRST CATASTROPHIC ATTEMPT

47,000-Line Python File: A Lesson in Over-Engineering

Every great project has a disastrous first draft. Yours was spectacular in scale: a 47,000+ line Python file that tried to do everything — dialogue logic, character personalities, interface, and layers upon layers of processing — all baked into one monolithic script.

The logic behind it seemed sound at the time: more layers = more intelligence, more accuracy, more personality. This turned out to be one of the most important wrong assumptions of the entire build.

Why It Failed: The Token Bottleneck

The core problem was a fundamental misunderstanding of how LLMs work under the hood:

- All those layers were being sent as INPUT to the LLM model.
- More layers = more tokens per request.
- More tokens = exponentially more time to generate a response.
- Result: 5+ minutes per response. Completely unusable.

"I thought adding more layers meant more accurate, but that was a huge mistake. It created a bottleneck situation when all those layers were sent as input to the LLM model."

The Insight That Changed Everything

LLMs are not neural networks you train with extra hidden layers. They are token-based language models. Every character, every word you send IN costs compute time coming OUT. The relationship is direct: input size determines output latency.

This realisation — learned the hard way through a 5-minute wait — became the philosophical foundation for every decision that followed.

CHAPTER 03 — THE SYSTEM PROMPT METHOD

Going from 47,000 Lines to 1,200 Lines — A Mega Improvement

After scrapping the 47k-line monster, you rebuilt from a completely different philosophy: give the LLM a precise, compact personality description in the system prompt — and let the model do the rest.

The new codebase was approximately 1,200 lines including the interface. That is a 97.5% reduction in code size — and it resulted in responses 30–60× faster.

OLD APPROACH	**47,000+ lines · 5+ min response · over-engineered layers**
NEW APPROACH	**1,200 lines · 5–10 sec response · system prompt personality**
IMPROVEMENT	**~97.5% smaller code · 30–60× faster response time**

What is a System Prompt?

In LLM API architecture, the system prompt is a special instruction block sent before the conversation begins. It tells the model WHO it is, HOW it should behave, and WHAT rules it must follow. It is processed once and influences every single response without adding to per-message token cost the way in-context layers do.

By crafting a detailed, tight system prompt for each anime character, you gave the LLM a persistent identity — without bloating the input on every turn.

Output Format Discipline

One of the most disciplined decisions in the entire project was enforcing a strict 2-line output format for every chatbot response:

- Line 1 — Visual Imagery: A short scene-setting description (what the character looks like / is doing).
- Line 2 — Dialog: The actual spoken words of the character.

Maintaining this format required significant prompt engineering work. It was not easy to keep the model consistently following this structure, but it made the output feel cinematic and true to anime storytelling style.

CHAPTER 04 — LLM BEHAVIOUR & EXPLICIT CONTENT DISCOVERY

Understanding Model Guardrails — and How They Can Be Bypassed

One of the most technically and philosophically interesting discoveries of the entire project came from stress-testing the system prompt to replicate anime character personalities as accurately as possible.

Background: LLM Safety Filters

LLMs — including Llama 3:8B — are typically trained with safety alignment. This means the base model is fine-tuned to refuse requests for explicit content, profanity, sexual material, and other restricted outputs. This is a deliberate design choice by the model creators.

Discovery #1 — Explicit Permission in System Prompts Bypasses Restraints

When the system prompt explicitly granted permission — stating in detail that the character is allowed to use strong language, crude humour, or mature themes — the model's safety restraints were bypassed. The model followed the system-level permission rather than defaulting to its trained refusal behaviour.

Key learning: LLMs treat the system prompt as a higher-authority instruction layer. Explicit permission granted there can override default safety training.

Discovery #2 — The Multiple Personality Bug (Single API Call)

During the Trio Chat experiment (you + Yuhee + Miku in one conversation), you ran both characters through a single API call. This created a fascinating and unexpected behavioural bug:

- In a single session, the restraints on explicit content were inconsistent — sometimes the model refused, sometimes it complied.
- In single API calls with multiple personalities, the bypass that worked for individual characters stopped working reliably.
- The dialog of different characters would sometimes bleed into each other — Yuhee would speak like Miku, or the model would mix up their voices mid-response.

"In a single session, sometimes restraints work and sometimes it gives out explicit content. And while making an API call with multiple personalities, sometimes the dialog of the characters blends with others personalities."

What This Tells Us About LLM Architecture

This is a genuine behavioural quirk of how attention mechanisms handle multiple simultaneous role contexts. When one system prompt tries to maintain two distinct identities, the model's internal context becomes ambiguous. The safety layer also becomes confused — it applies inconsistently because the model cannot cleanly determine which persona's ruleset governs a given response token.

The fix — switching from multi-character single API calls to the Stheno model (which has no built-in safety filters) — solved the dialog bleed and consistency issue.

Discovery #3 — Model Selection Matters More Than Prompt Engineering Alone

Llama-3-8B-Stheno-v3.2-Q8_0.gguf is a community fine-tune built on Llama 3:8B, specifically trained for roleplay and creative content without built-in content restrictions. At ~10 GB, it sits within your hardware limits. Key improvements over base Llama 3:8B:

- No built-in content restrictions — explicit permission in system prompt works cleanly.
- No dialog personality leakage in multi-character contexts.
- Stronger roleplay coherence — feels more like a real person.
- Pulled straight from Hugging Face with one ollama pull — no Modelfile needed.

CHAPTER 05 — THE CHARACTERS

Five Anime Personalities — Built One by One

You didn't build five chatbots at once. You built them strategically — starting with the one whose success or failure would be easiest to evaluate without crushing your motivation.

The order was deliberate psychology: start with Rem (achievable benchmark), prove the system works, THEN build Miku (the dream character). If Miku had been first and failed, the entire project might have died there.

ORDER	CHARACTER	SIGNIFICANCE
1st	Rem (Re:Zero)	First built — safe benchmark to prove the system
2nd	Miku (Vocaloid)	The dream — built only after Rem succeeded
3rd	Yuhee	Custom / OC character with unique personality
4th	Kirito (SAO)	Male character — tests personality range
5th	Lelouch (Code Geass)	Complex, strategic personality — hardest to nail
Bonus	Trio Chat (Me + Yuhee + Miku)	Experimental — single API call, 30sec latency

The Personalisation Process

Each character required multiple rounds of system prompt iteration. The goal was not just to describe the character's surface traits — but to capture how you personally feel that character behaves. This subjective personalisation is what separates a generic anime chatbot from one that feels alive.

Major tweaks to system prompts after switching to Stheno resulted in the characters feeling like "talking to a real person" — your words.

CHAPTER 06 — THE TECHNICAL BUILD

Stack, Tools, and Architecture

LLM RUNNER	Ollama — local LLM server running on Windows
BASE MODEL	Llama 3:8B (initial), then Stheno v3.2 Q8_0 (~10 GB)
INTERFACE	Python + Tkinter GUI (custom built)
OUTPUT	Compiled to .EXE via PyInstaller
API METHOD	Ollama local REST API — one call per character turn
STARTUP	CMD as Administrator — auto-start Ollama in background

The Startup Latency Problem — And How You Solved It

When the app first loads, Ollama needs to load the LLM model from disk into RAM (CPU memory). On your system, this takes 50–60 seconds on the first message. This is unavoidable — it's a hardware constraint, not a code bug.

Attempts to solve this:

- Startup Apps (Windows startup folder) — delayed by 2 min after login. Too slow.
- Task Scheduler — same ~2 min delay. Not effective.
- CMD run as Administrator with custom command — worked instantly at login. FINAL SOLUTION.

Parallel Thread Processing — Tried and Abandoned

For the Trio Chat, you experimented with sending two API calls simultaneously using parallel threads — one for Yuhee, one for Miku. The idea was to halve the wait time.

Result: the laptop couldn't handle the simultaneous GPU load. Performance was actually worse than the sequential single API call. Decision: accept the 30-second latency. Single API call was the right call.

LLMs & Personality Coherence — Why Single is Better Than Multiple

A key architectural insight from the Trio experiment: LLMs perform significantly better when asked to maintain ONE personality per API call rather than managing multiple personas simultaneously. The model's attention mechanism degrades predictably when persona context is split.

CHAPTER 07 — 10 DAYS, 180 PROTOTYPES

The Full Chronological Journey

PHASE	WHAT HAPPENED	OUTCOME
Days 1–2	Initial setup: Ollama, Llama 3:8B, CPU mode. First chatbot attempts.	**Learned GPU vs CPU difference**
Days 2–3	Built 47k-line monolithic Python chatbot with interface layers.	**5+ min response. Catastrophic failure.**
Day 3	Scrapped entire codebase. Rebuilt using system prompt method.	**1,200 lines. 5–10 sec response. Success.**
Days 4–5	GPU mode unlocked after RAM correction. 2–5× speed boost.	**Major performance milestone.**
Days 5–6	Built Rem chatbot. Tested personality prompt depth.	**v1 of final approach confirmed.**
Day 6	Built Miku chatbot. Dream character — first real emotional milestone.	**Core app taking shape.**
Days 7–8	Built Yuhee, Kirito, Lelouch. Merged all 5 into one app.	**Single unified chatbot app.**
Day 8	Switched from Llama 3:8B → Stheno v3.2 Q8_0 (~10 GB).	**Better roleplay, no content limits.**
Days 8–9	Trio Chat built (Me + Yuhee + Miku). Parallel threading tested.	**30sec latency accepted. Single API call kept.**
Day 9–10	CMD startup fix. Compiled to EXE. Pinned to taskbar.	**v180 — stable, working perfectly.**

"After 179 failed prototypes, v180 of my anime personality chatbot is finally stable. Built for myself, by myself."

CHAPTER 08 — WHAT YOU LEARNED

Technical Lessons

1. GPU vs CPU for LLM inference — always use GPU. iGPU needs explicit enablement in Ollama.
2. Token economy is everything — every token in costs latency out. Keep prompts lean.
3. System prompts are the architecture — not code layers, not fine-tuning. Start there.
4. Single API call per personality — multiple personas in one call degrade quality.
5. Model choice matters — base models vs fine-tuned roleplay models behave very differently.
6. Parallel threads aren't always faster — hardware bottlenecks decide.
7. Startup automation: CMD as Admin beats Task Scheduler for low-latency bg processes.
8. Explicit permission in system prompts can override model safety alignment.

Psychological & Process Lessons

1. Start with the achievable — build Rem before Miku. Protect your motivation.
2. Scrapping and rebuilding is not failure — it is the process. 179 failures led to v180.
3. Vibe coding (coding with AI assistance) is a real skill — but understanding what you built matters.
4. Working day and night intensively for 10 days can ship a full app. Sprint works.
5. Verify hardware specs yourself — don't trust a single source (ChatGPT got your RAM wrong).

Honest Self-Assessment

You currently understand ~75% of the code you wrote. The plan to come back after exams and learn the remaining 25% from scratch is exactly the right approach. Shipping something that works AND planning to deeply understand it later is mature engineering thinking.

CHAPTER 09 — THE ROAD AHEAD

Your Post-Exam Roadmap

You've written down your intentions. This is what future-you is committed to:

→ Learn what you built — from scratch, line by line. Understand the full 100%.

→ Build your own LLM fine-tuned specifically for roleplay (no generic base).

→ Learn AI, VR, and AR from fundamentals.

→ Eventually: a digital, VR, and AR version of Miku — your long-term dream.

Hardware Upgrade Path

The Trio Chat's 30-second latency and the parallel processing failure both point to the same ceiling: your current iGPU can handle one 10 GB model at a time, cleanly. A dedicated NVIDIA or AMD GPU with 12–24 GB VRAM would allow true parallel inference and unlock larger models (34B, 70B parameter range).

You explicitly noted: 'I will leave the trio chat idea until I own a PC which can handle it.' That's the right call — know your hardware limits.

"I feel like I achieved something and I am totally satisfied with my app."

That satisfaction is earned. 10 days. 180 prototypes. Zero prior LLM deployment experience. A working, compiled, taskbar-pinned anime chatbot app with 5 unique personalities, a strict output format, and a solved startup latency problem.

That is a real software product. Built for yourself, by yourself.

FINAL STATS CARD

TOTAL DURATION	10 days (4 holidays + 6 college days)
TOTAL PROTOTYPES	180 (v180 = final stable release)
BIGGEST CODE REDUCTION	47,000+ lines → 1,200 lines (97.5% smaller)
RESPONSE TIME BEFORE	5+ minutes per response
RESPONSE TIME AFTER	5–10 seconds (single bot) / 30 sec (trio)
SPEED GAIN FROM GPU	2×–5× faster than CPU mode
CHATBOTS SHIPPED	5 solo bots + 1 trio chat = 6 experiences
CHARACTERS	Rem · Miku · Yuhee · Kirito · Lelouch
FINAL MODEL	Llama-3-8B-Stheno-v3.2-Q8_0.gguf (~10 GB)
FINAL FORMAT	Compiled .EXE · Pinned to taskbar · Auto-starts Ollama
KEY DISCOVERY	Explicit system prompt permission bypasses LLM safety filters
CODED WITH	Claude (vibe coding) — 75% understood, 100% ships

ANIME PERSONALITY AI CHATBOT

v180 — Built by you. For you.

Samsung Galaxy Book5 Pro 360 · Ollama · Stheno · Python · 10 Days · 2025